Master's Thesis Exposé
DrEureka-Based Sim2Real Reinforcement Learning for the AR4 Robot

Fabian Kontor
University of Heidelberg
f.kontor@stud.uni-heidelberg.de

1 Motivation

Training robots in real life can be cumbersome and time consuming. To alleviate these concerns and enable robotics at scale, we will pursue a method called Sim2Real to train an AR4 robot first in simulation, and then try to transfer the learned skills zero-shot to the real world.

2 Objectives

This thesis will develop and evaluate a Unity-based Sim2Real reinforcement learning pipeline for the AR4 robot, incorporating DrEureka-style reward generation and domain randomization (Ma, Liang, H. Wang, et al. 2024). The work will begin with a baseline task to validate the reinforcement learning setup. Task complexity will then be gradually increased to test the robustness of the learned policies under more challenging conditions. In preparation for Sim2Real transfer, we will use large language models to automatically generate reward functions and compare their effectiveness against manually designed configurations.

Task complexity will scale in this progression:

1. **T0 - Sanity Check Task:** Move the end effector to a fixed target position.
2. **T1 - Static Pick Task:** Reach and grab a fixed cube.
3. **T2 - Pick-and-Place:** Pick up a cube from a fixed location and transfer it to another fixed drop zone.
4. **T3 - Pick-and-Place (randomized positions):** Pick and place from randomized (but bounded) positions.
5. **T4 - Pick-and-Place (camera-based input):** Pick and place, but state input is replaced with raw camera input in preparation for Sim2Real.
6. **T5 - Multi-object Stack (optional):** Stack multiple differently colored cubes.

We will evaluate Sim2Real performance for tasks T4 and T5. Due to time and complexity constraints, we will be omitting the domain randomization aspect of DrEureka and will try to fine-tune physical parameters manually. In the original paper by (Ma, Liang, H. Wang, et al. 2024), removing domain randomization in the forward locomotion task led to a performance drop from 1.83 to 1.21 meters pser second in forward velocity, and from 5.00 to 4.17 meters in distance traveled.

3 Related Work

Reinforcement learning (RL) enables autonomous skill acquisition without explicit supervision (Sutton et al. 2018). It's benefit is that policies can be learned without human supervision, by exploration of an agent in an environment. However, designing effective reward functions remains a key challenge, often requiring extensive domain knowledge. Recent works such as (Ma, Liang, G. Wang, et al. 2023) and (Ma, Liang, H. Wang, et al. 2024) propose using large language models (LLMs) to automate this process. They also propose an LLM-guided domain randomization process, enhancing Sim2Real performance.

We will implement similar reward design automation but focus solely on reward function refinement. We will also adopt Proximal Policy Optimization (Schulman et al. 2017), the same RL algorithm used in DrEureka.

4 Methodology

We will be using the existing AR4 Unity simulation. Unity is a good fit, since the AR4 robot is already implemented and Unity has a very versatile and realistic physics engine. The training loop will use Stable-Baselines3 (SB3) via Unity's Gym

wrapper, enabling RL experiments from Python using PPO, SAC, or TD3.

To enable reward function reflection, we will build a custom Python logger capturing:

- Observation statistics (mean, std, min, max)
- Reward breakdown by component
- Episode success rates, loss, and entropy

This metadata will be fed into a state-of-the-art LLM to design improved reward functions. We will use structured prompts that describe the current training status (e.g., observation statistics, episode rewards, and policy metrics) and ask the LLM to suggest modified or new reward components to improve performance. Prompts will also include reminders to prioritize safety and stability, especially in preparation for real-world deployment. A python script will then inject the new reward function, recompile if necessary, and restart the training loop, creating an iterative loop.

Since the LLM is only called periodically for reward function generation (e.g., every N training iterations), rather than per-step or per-action, the overall API cost remains low and bounded.

For real-world deployment, we will use the AR4 Python API, which allows sending joint commands. SB3 models (exported as ONNX) will be mapped directly to joint control commands.

Each task (T0–T4) will be evaluated using both human- and LLM-designed rewards to measure training efficiency, task success, and, in the case of T4, transfer quality.

5 Work Packages

To manage complexity and minimize risk, we will be splitting our work into five work packages:

- **WP1**: RL pipeline setup and T0 baseline
- **WP2**: Task difficulty scaling and evaluation up to T3
- **WP3**: DrEureka-style LLM-based reward design
- **WP4**: Vision-based task T4 and (optional) T5
- **WP5 (Stretch)**: Real-world deployment and transfer evaluation

6 Timeline

- Month 1: Literature review, Unity baseline setup and training for T0
- Month 2–3: Train tasks T1–T3 with human rewards
- Month 4: Implement LLM-based reward design (WP3)
- Month 5: Vision-based input (T4), optional real-world deployment
- Month 6: Final evaluations, writing the thesis

7 Resources

- **Unity:** unity.com
- **Stable-Baselines3:** on GitHub
- **Python:** python.org
- **ONNX:** onnx.ai
- **AR4 Robot:** anninrobotics.com/ar4
- **LiteLLM:** github.com/BerriAI/litellm
- **AutoHotkey (for workflow automation):** autohotkey.com

References

Ma, Yecheng Jason, William Liang, Guanzhi Wang, et al. (2023). "Eureka: Human-Level Reward Design via Coding Large Language Models". In: *arXiv* 2310.12931. Equal advising by Linxi "Jim" Fan and Anima Anandkumar. URL: https://arxiv.org/abs/2310.12931.

Ma, Yecheng Jason, William Liang, Hungju Wang, et al. (2024). "DrEureka: Language Model Guided Sim-To-Real Transfer". In: *Proceedings of Robotics: Science and Systems (RSS)*. Equal contribution by Jason Ma and William Liang.

Schulman, John et al. (2017). "Proximal Policy Optimization Algorithms". In: *arXiv* 1707.06347. URL: https://arxiv.org/abs/1707.06347.

Sutton, Richard S et al. (2018). *Reinforcement Learning: An Introduction*. MIT Press.